利用栅栏实现同步

Posted on 2023-11-12 15:10:03 by 恋恋风辰 | In C++ | 原文链接 (/articlepage?id=2Y47TJQuXozDbR4mcrMmZ0XwMM2)

简介

前文我们通过原子操作实战实现了无锁队列，今天完善一下无锁的原子操作剩余的知识，包括Relaese和Acquire内存序在什么情况下是存在危险的，以及我们可以利用栅栏机制实现同步等等。

线程可见顺序

我们提到过除了 `memory_order_seq_cst` 顺序，其他的顺序都不能保证原子变量修改的值在其他多线程中看到的顺序是一致的。

但是可以通过同步机制保证一个线程对原子变量的修改对另一个原子变量可见。通过“Syncronizes With” 的方式达到先行的效果。

但是我们说的先行是指 “A Syncronizes With B ”， 如果A 的结果被B读取，则A 先行于B。

有时候我们线程1对A的store操作采用release内存序，而线程2对B的load采用acquire内存序，并不能保证A 一定比 B先执行。因为两个线程并行执行无法确定先后顺序，我们指的先行不过是说如果B读取了A操作的结果，则称A先行于B。

我们看下面的一段案例

```
#include <iostream>
#include <atomic>
#include <thread>
#include <cassert>
std::atomic<bool> x, y;
std::atomic<int> z;

void write_x()
{
    x.store(true, std::memory_order_release); //1
}
void write_y()
{
    y.store(true, std::memory_order_release); //2
}
void read_x_then_y()
{
    while (!x.load(std::memory_order_acquire));
    if (y.load(std::memory_order_acquire))   //3
        ++z;
}
void read_y_then_x()
{
    while (!y.load(std::memory_order_acquire));
    if (x.load(std::memory_order_acquire))   //4
        ++z;
}
```

我们写一个函数测试，函数TestAR中初始化x和y为false， 启动4个线程a,b,c,d，分别执行write_x, write_y, read_x_then_y, read_y_then_x.

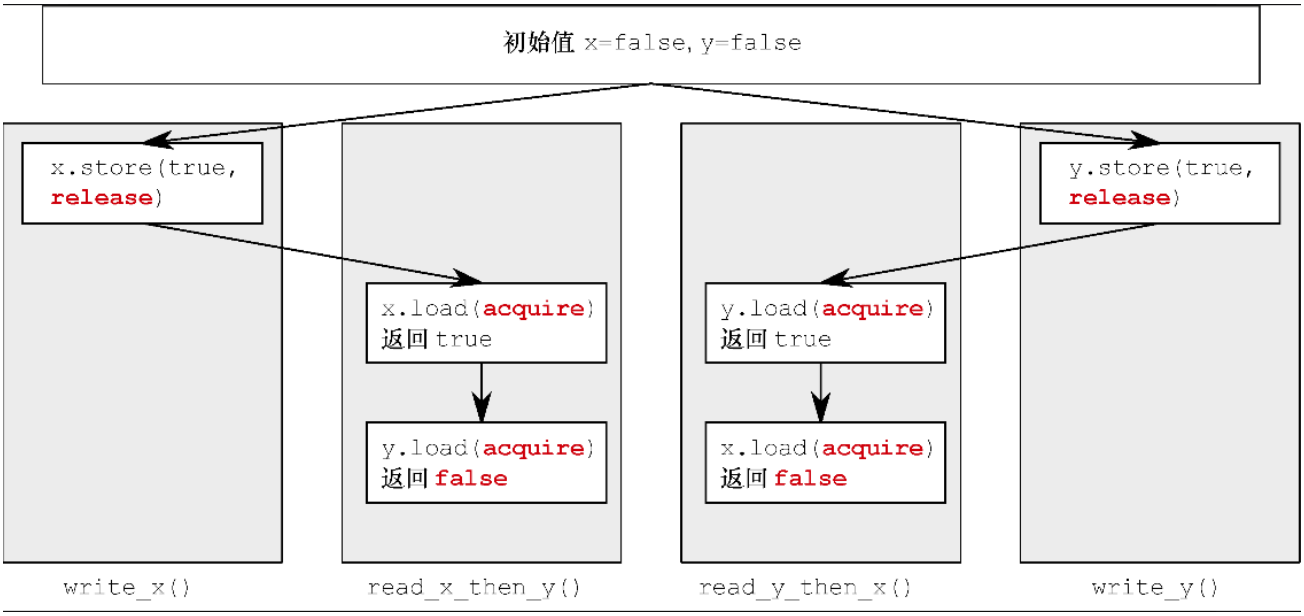
```
1. void TestAR()
2. {
3.     x = false;
4.     y = false;
5.     z = 0;
6.     std::thread a(write_x);
7.     std::thread b(write_y);
8.     std::thread c(read_x_then_y);
9.     std::thread d(read_y_then_x);
10.    a.join();
11.    b.join();
12.    c.join();
13.    d.join();
14.    assert(z.load() != 0); //5
15.    std::cout << "z value is " << z.load() << std::endl;
16. }
```

有的读者可能会觉5处的断言不会被触发，他们认为c和d肯定会有一个线程对z执行++操作。他们的思路是这样的。
1 如果c线程执行read_x_then_y没有对z执行加加操作，那么说明c线程读取的x值为true, y值为false。
2 之后d线程读取时，如果保证执行到4处说明y为true，等d线程执行4处代码时x必然为true。
3 他们的理解是如果x先被store为true，y后被store为true，c线程看到y为false时x已经为true了，那么d线程y为true时x也早就为true了，所以z一定会执行加加操作。

上述理解是不正确的，我们提到过即便是releas和acquire顺序也不能保证多个线程看到的一个变量的值是一致的，更不能保证看到的多个变量的值是一致的。

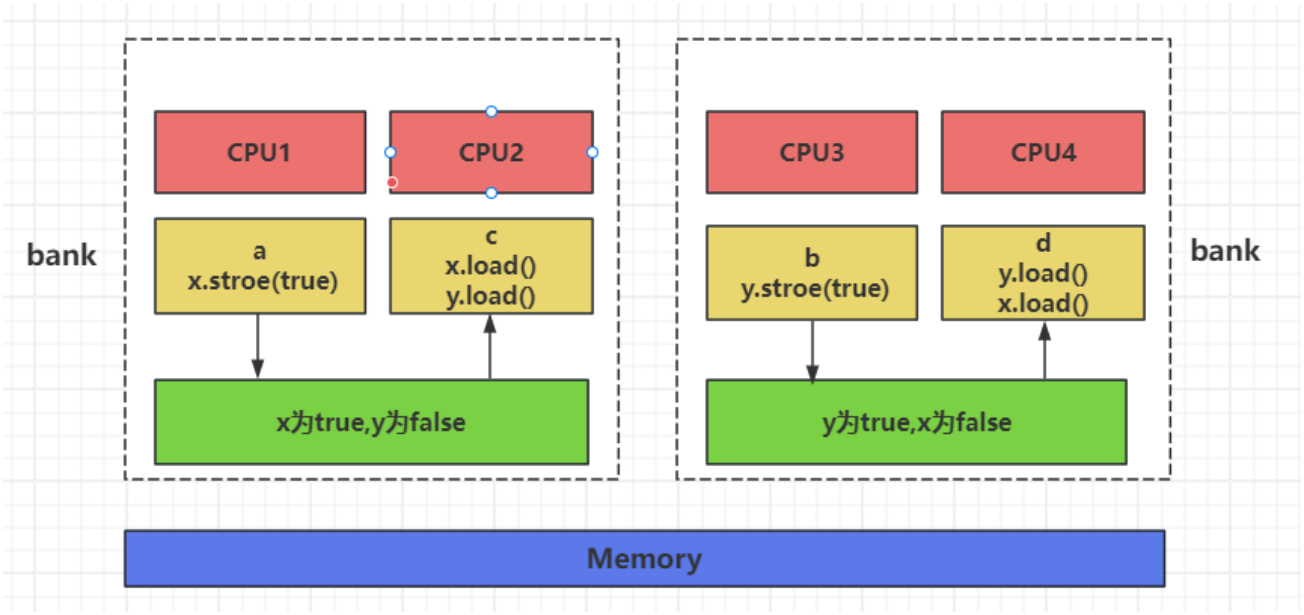
变量x和y的载入操作3和4有可能都读取false值（与宽松次序的情况一样），因此有可能令断言触发错误。变量x和y分别由不同线程写出，所以两个释放操作都不会影响到对方线程。

看下图



无论x和y的store顺序谁先谁后，线程c和线程d读取的x和y顺序都不一定一致。

从CPU的角度我们可以这么理解



在一个4核CPU结构的主机上，a,b,c,d分别运行在不同的CPU内核上。

a执行 `x.store(true)` 先被线程c读取，而此时线程b对y的store还没有被c读取到新的值，所以此时c读取的x为true，y为false。

同样的道理，d可以读取b修改y的最新值，但是没来的及读取x的最新值，那么读取到y为true，x为false。

即使我们采用release和acquire方式也不能保证全局顺序一致。如果一个线程对变量执行release内存序的store操作，另一个线程不一定会马上读取到。这个大家要理解。

栅栏

有时候我们可以通过栅栏保证指令编排顺序。

看下面一段代码

```
1. #include <atomic>
2. #include <thread>
3. #include <assert.h>
4. std::atomic<bool> x,y;
5. std::atomic<int> z;
6. void write_x_then_y()
7. {
8.     x.store(true,std::memory_order_relaxed); // 1
9.     y.store(true,std::memory_order_relaxed);   // 2
10. }
11. void read_y_then_x()
12. {
13.     while(!y.load(std::memory_order_relaxed));  // 3
14.     if(x.load(std::memory_order_relaxed))  // 4
15.         ++z;
16. }
17. int main()
18. {
19.     x=false;
20.     y=false;
21.     z=0;
22.     std::thread a(write_x_then_y);
23.     std::thread b(read_y_then_x);
24.     a.join();
25.     b.join();
26.     assert(z.load()!=0);  //5
27. }
```

上面的代码我们都采用的是 `memory_order_relaxed` ，所以无法保证a线程将x,y修改后b线程看到的也是先修改x，再修改y的值。b线程可能先看到y被修改为true，x后被修改为true，那么b线程执行到4处时x可能为false导致z不会加加，5处断言会被触发。

那我们之前做法可以解决这个问题

```
1. void write_x_then_y3()
2. {
3.     x.store(true, std::memory_order_relaxed); // 1
4.     y.store(true, std::memory_order_release);   // 2
5. }
6.
7. void read_y_then_x3()
8. {
9.     while (!y.load(std::memory_order_acquire));  // 3
10.    if (x.load(std::memory_order_relaxed))  // 4
11.        ++z;
12. }
```

可以通过 `std::memory_order_release` 和 `std::memory_order_acquire` 形成同步关系。

线程a执行 `write_x_then_y3` ，线程b执行 `read_y_then_x3` ，如果线程b执行到4处，说明y已经被线程a设置为true。

线程a执行到2，也必然执行了1，因为是 `memory_order_release` 的内存顺序，所以线程a能2操作之前的指令在2之前被写入内存。

同样的道理，线程b在3处执行的是 `memory_order_acquire` 的内存顺序，所以能保证4不会先于3写入内存，这样我们能知道1一定先行于4.

进而推断出z会加加，所以不会触发 `assert(z.load() != 0);` 的断言。

其实我们可以通过栅栏机制保证指令的写入顺序。栅栏的机制和 `memory_order_release` 类似。

```
1. void write_x_then_y_fence()
2. {
3.     x.store(true, std::memory_order_relaxed);  //1
4.     std::atomic_thread_fence(std::memory_order_release);  //2
5.     y.store(true, std::memory_order_relaxed);  //3
6. }
7.
8. void read_y_then_x_fence()
9. {
10.     while (!y.load(std::memory_order_relaxed));  //4
11.     std::atomic_thread_fence(std::memory_order_acquire); //5
12.     if (x.load(std::memory_order_relaxed))  //6
13.         ++z;
14. }
```

我们写一个函数测试上面的逻辑

```
1. void TestFence()
2. {
3.     x = false;
4.     y = false;
5.     z = 0;
6.     std::thread a(write_x_then_y_fence);
7.     std::thread b(read_y_then_x_fence);
8.     a.join();
9.     b.join();
10.     assert(z.load() != 0);   //7
11. }
```

7处的断言也不会触发。我们可以分析一下，

线程a运行 `write_x_then_y_fence` ，线程b运行 `read_y_then_x_fence` .

当线程b执行到5处时说明4已经结束，此时线程a看到y为true，那么线程a必然已经执行完3.

尽管4和3我们采用的是 `std::memory_order_relaxed` 顺序，但是通过逻辑关系保证了3的结果同步给4，进而”3 happens-before 4”

因为我们采用了栅栏 `std::atomic_fence` 所以，5处能保证6不会先于5写入内存，(memory_order_acquire保证其后的指令不会先于其写入内存)

2处能保证1处的指令先于2写入内存，进而”1 happens-before 6”, 1的结果会同步给 6

所以”atomic_thread_fence”其实和”release-acquire”相似，都是保证 `memory_order_release` 之前的指令不会排到其后， `memory_order_acquire` 之后的指令不会排到其之前。

总结

视频链接

https://space.bilibili.com/271469206/channel/collectiondetail?sid=1623290 (https://space.bilibili.com/271469206/channel/collectiondetail?sid=1623290)

源码链接

https://gitee.com/secondtonone1/boostasio-learn/tree/master/concurrent/day13-fence (https://gitee.com/secondtonone1/boostasio-learn/tree/master/concurrent/day13-fence)

浏览(1610) (JavaScript:void(0);) 评论(0) (JavaScript:void(0);) ♡喜欢(517) (JavaScript:void(0);)

热门评论

请输入正文

评论

没有更多评论了,点击收起 ≪ (JavaScript:void(0);)

热门文章

slice介绍和使用

♡喜欢(521) 浏览(1546)

C++ 类的继承封装和多态

♡喜欢(588) 浏览(2196)

windows环境搭建和vscode配置

♡喜欢(587) 浏览(1595)

解密定时器的实现细节

♡喜欢(566) 浏览(1449)

Linux环境搭建和编码

♡喜欢(594) 浏览(5076)

最新评论

C++ 类的继承封装和多态

gzy:谢谢老师，写的真的很好，本来对这些新的关键字都很模糊，几篇下来配合着ai记着笔记一下子就清晰了

C++ 右值引用与移动构造函数

李程浩:1

C++ 线程管控

secondtonone1:好的，已修复，写的时候马虎了，谢谢指正。

线程基础

secondtonone1:谢谢，笔误已修复

再谈单例模式

secondtonone1:是的，C++11以后返回局部static变量对象能保证线程安全了。

线程安全的无锁栈

secondtonone1:谢谢支持，如果pop的次数大于push的次数是会让线程处于重试的，这个是测试用例，必须满足push和pop的次数相同，实际情况不会这么使用。栈的设计没有问题。

创建项目和编译

secondtonone1:谢谢支持

visual studio配置boost库

secondtonone1:是的，您说的是对的，windows平台要采用msvc方式。

C++ 类的拷贝构造、赋值运算、单例模式

secondtonone1:好的，已修复。

asio多线程模式IOThreadPool

secondtonone1:这么优秀吗

asio实现http服务器

secondtonone1:我认为IO复用节省了线程资源，不用像传统服务器一样为每个连接单独创立线程，而是统一通过epoll或者iocp通知。

类和对象

陈宇航:支持！！！！

解决博客回复区被脚本注入的问题

secondtonone1:走到现在我忽然明白一个道理，无论工作也好生活也罢，最重要的是开心，即使一份安稳的工作不能给我带来事业上的积累也要合理的舍弃，所以我还是想去做喜欢的方向。

泛型算法的定制操作

secondtonone1:lambda和bind是C11新增的利器，善于利用这两个机制可以极大地提升编程安全性和效率。

双链表实现LRU算法

secondtonone1:双链表插入和删除节点是本篇的难点，多多练习即可。

构造函数

secondtonone1:构造函数是类的基础知识，要着重掌握

面试题汇总(一)

secondtonone1:看到网络上经常提问的go的问题，做了一下汇总，结合自己的经验给出的答案，如有纰漏，望指正批评。

使用mongo

secondtonone1:mongo操作是程序员必备的基础技能，包括增删改查，以及较为复杂的查询，分组查询，条件查询，模糊查询，文档内查询等，本文结合自己实际开发经验做了整理。

interface应用

secondtonone1:interface是万能类型，但是使用时要转换为实际类型来使用。interface丰富了go的多态特性，也降低了传统面向对象语言的耦合性。

golang 函数介绍

secondtonone1:函数是go中的一等公民，作为新兴语言，go摒弃了面向对象的一些糟粕，采取接口方式编程，而接口方式编程都是基于函数的，参数为interface，进而达到泛型作用，比如sort排序，只需要传入的参数满…

堆排序

secondtonone1:堆排序非常实用，定时器就是这个原理制作的。

Linux环境搭建和编码

恋恋风辰:Linux环境下go的安装比较简单，可以不用设置GOPATH环境变量，后期我们学习go mod 之后就拜托了go文件目录的限制了。

slice介绍和使用

恋恋风辰:切片作为引用类型极大的提高了数据传递的效率和性能，但也要注意切片的浅拷贝贝隐患，算是一把双刃剑，这世间的常态就是在两极之间寻求一种稳定。

其他链接

GitHub (https://github.com)
个人博客 (https://www.limerence2017.com)
博客园 (https://www.cnblogs.com/secondtonone1/)
CSDN (https://blog.csdn.net/secondtonone1)

B站



个人微信



个人公众号



本站累计访问量204866次

本网站内容全部为原创，谢绝转载。
京公网安备 11010502047625号 (http://www.beian.gov.cn/portal/registerSystemInfo?recordcode=11010502047625)
京ICP备2021033081号 (https://beian.miit.gov.cn/)